Pruning for Efficient LLM Serving

AJ Seo, Cindy Liu
Carnegie Mellon University

Introduction and Overview

Why prune LLMs?

- LLMs are expensive to serve: high **memory footprint** and **inference latency**
- Pruning removes weights to reduce cost **without retraining**

Two pruning strategies:

- **Unstructured** — removes individual weights; strong compression but no real speedup without specialized kernels
- **Structured** — removes entire heads/neurons; deployment-friendly but can hurt quality

Our goal:

- Compare both strategies on **OPT-1.3B** in a unified setup
- Identify practical operating points balancing **quality**, **latency**, and **memory**

Problem Statement

- How do unstructured and structured pruning compare at matched sparsity levels?
- At what sparsity threshold does quality degradation become unacceptable for our target tasks?
- Which methods deliver real end-to-end inference speedups rather than only parameter-count reduction?

Related Works

Unstructured Pruning

- **Han et al., 2015:** Individual weights pruned with minimal accuracy loss
- **Lottery Ticket (Frankle & Carlin, 2019):** Sparse subnetworks preserve trainability in LLMs like BERT
- **SparseGPT (Frantar & Alistarh, 2023):** One-shot post-training pruning for LLMs with limited quality loss

Structured Pruning

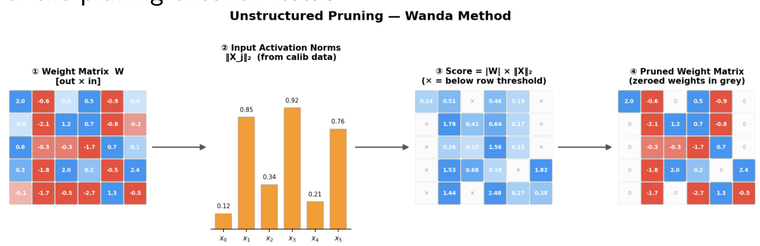
- **Network Slimming (Liu et al., 2017):** Channel-level sparsity compatible with dense kernels
- **LLM-Pruner (Ma et al., 2023):** Structured pruning on LLMs for practical inference speedups

Key Takeaway

- Unstructured pruning → stronger compression at a given perplexity
- Structured pruning → produces more predictable wall-clock speedups by removing full computational weights *(Cheng et al., 2023; He & Xiao, 2024)*

Methods

We compared two pruning strategies on OPT-1.3B using 128 calibration sequences from WikiText-2, evaluating perplexity, latency, throughput, and peak GPU memory at 0/20/40/60/70% sparsity. The experiment was run with Google Colab T4 GPU (16GB GDDR6, 320 GB/s memory bandwidth, 8.1 TFLOPS FP32). Note that no post-pruning fine-tuning was applied, isolating the effect of the pruning criterion itself.

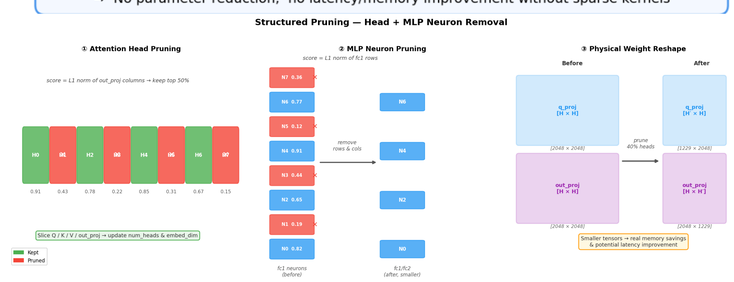


Unstructured Pruning (Wanda)

$$\text{score}_{i,j} = |W_{i,j}| \times \|X_j\|_2$$

For each output row i: zero out the $k = \lfloor n_{\text{in}} \times s \rfloor$ weights with the lowest score (s = target sparsity)

Tensor shape is UNCHANGED — zeroed weights remain in memory as 0.0
→ No parameter reduction, no latency/memory improvement without sparse kernels

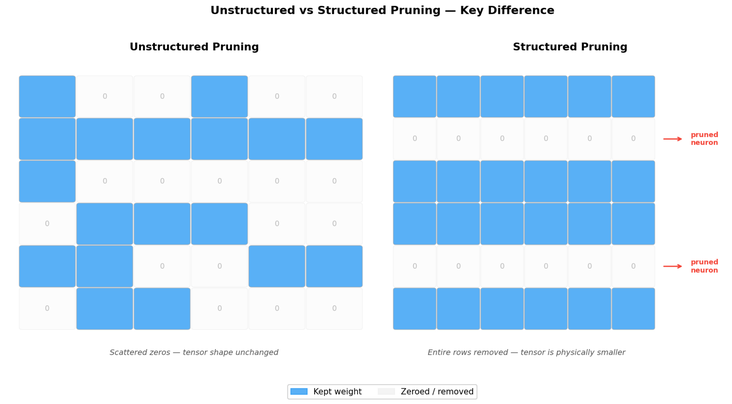


Structured Pruning — Head + MLP Neuron Removal

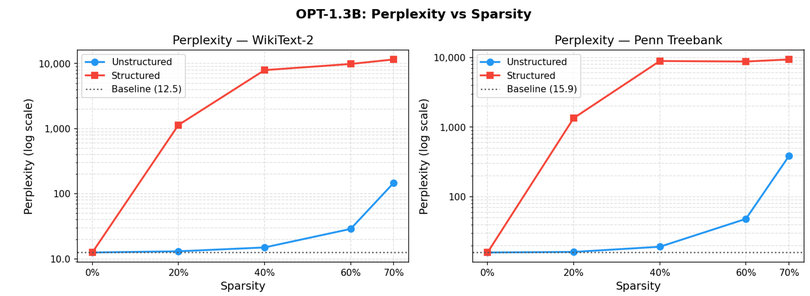
Attention heads: $\text{score}_h = \sum_{i,d} |W_{\text{out}}[i,\ h \cdot d_h : (h+1) \cdot d_h]|$ → remove lowest-scoring heads, slice Q / K / V / out_proj, update num_heads & embed_dim

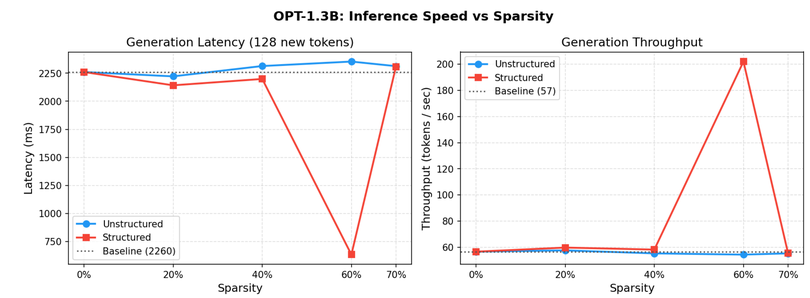
MLP neurons: $\text{score}_n = \sum_j |W_{\text{fc1}}[n, j]|$ → remove lowest-scoring neurons, slice fc1 rows & fc2 columns

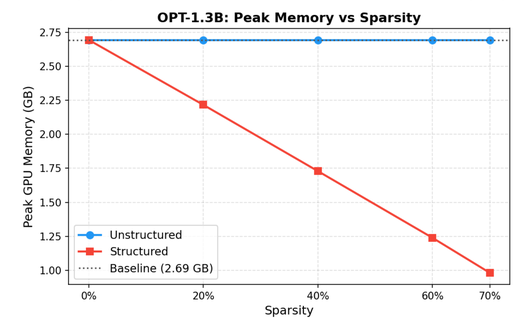
Both operations physically shrink tensors → real parameter reduction & memory savings
Quality collapses without fine-tuning; latency improvement is non-monotonic (hardware tile alignment)



Results and Evaluation







Key findings.

- **Unstructured pruning preserves quality but not runtime efficiency.** Perplexity stays near baseline up to moderate sparsity because individual weights are highly redundant, but tensor shapes remain unchanged, so memory and latency do not improve without sparse kernels.
- **Structured pruning reduces memory by physically shrinking the model.** Removing full attention heads and MLP neurons creates smaller tensors, so parameter count and peak GPU memory decrease approximately linearly with sparsity.
- **Structured pruning hurts quality without recovery training.** Perplexity collapses even at low sparsity, likely because magnitude-based pruning removes important heads or neurons without using stronger saliency metrics or post-pruning fine-tuning.
- **Practical speedups depend on hardware alignment, not just sparsity.** Structured pruning produced a large latency improvement at 60% sparsity (with reruns) but not at 70%, suggesting that inference speed depends on GPU-friendly matrix dimensions as well as compression level.